

Detailed & Extended Comparison — TsDiff vs NsDiff

Prédiction et calibration probabiliste, à budget d'échantillonnage égal — régimes *daily*, *weekly-natif* et *mensuel*

DeepEdgeBenchmark · version détaillée du duel · 2026-08-06 · chaque terme technique défini, expliqué, illustré et **vérifié à la source dans le code** (fichier:ligne cités)

Comment lire ce document. Il reprend le résumé du duel NsDiff vs TSDiff et le déplie : d'abord le verdict et le mécanisme (§1–§3), puis un **glossaire étendu** (§4) où chaque mot technique reçoit une *définition*, une *explication détaillée* et un *exemple spécifique*. Partout où une notion vient d'un calcul, elle est rattachée à la fonction exacte du dépôt qui la produit — parce que, comme l'a montré le cas du « p », les formulations en prose des notes sont parfois imprécises alors que le code, lui, est univoque.

0. Cadre — et où cette étude se situe

Deux modèles génératifs de prévision de prix sont comparés : **NsDiff** et **TSDiff**. Tous deux produisent, pour chaque prévision, un *nuage* de trajectoires de prix futurs ; l'intervalle de prédiction à 95 % est lu comme les quantiles empiriques 2,5 % / 97,5 % de ce nuage. La comparaison est faite à **budget d'échantillonnage égal** (200 tirages des deux côtés), sur des **prix identiques gelés**, avec la **même formule de lecture** et la **même machinerie de test**, sur trois régimes : *daily* (un modèle quotidien poussé jusqu'à la cible), *weekly-natif* (entraîné sur rendements hebdomadaires) et *mensuel*.

Le verdict est net et se répète : **en weekly et en mensuel, NsDiff domine ; en daily, les deux modèles sont à égalité sur le point mais NsDiff domine largement la fiabilité de l'incertitude**. Le gain de NsDiff se loge dans la *calibration*, pas dans l'erreur ponctuelle.

La big picture — ce que cette étude ne dit PAS. Le duel prédiction/calibration est une étape importante, mais *une seule étape*. La calibration statistique ne dit rien de la **valeur économique** : un modèle peut être parfaitement calibré et sans aucun *edge* pour trader (des intervalles larges et bien calibrés ne rapportent rien). L'étape manquante est un **backtest économique** : transformer les intervalles en décisions (sizing de position, limites VaR, signaux), inclure coûts de transaction (0 € pour l'instant) et slippage, mesurer PnL / Sharpe / drawdown — NsDiff contre GARCH sur ce terrain. C'est le seul test qui tranche « utilisable en trading ». S'y ajoutent des écarts opérationnels : le protocole *train-once-forward* (GARCH est refit à chaque origine — en production il faudrait définir la cadence de refit de NsDiff), la config de production visée est l'**ensemble 5 graines à 200+ tirages** (le meilleur rapport qualité/prix, cf. §4.7), et le dashboard **oos** tourne encore en *single-seed* / 50 tirages, donc non représentatif.

Sommaire

0. Cadre et big picture

1. Le verdict, les trois régimes

1.1 Weekly-natif

1.2 Daily

1.3 Mensuel

1.4 Bilan des 30 cellules

2. Le mécanisme (paramètres / données)

3. Deux erreurs de protocole

4. Glossaire étendu

- 4.1 Modèles, architecture, diffusion
- 4.2 Régimes, horizons, origines, données
- 4.3 Budget, intervalles, quantiles
- 4.4 Les métriques
- 4.5 Les tests statistiques

- 4.6 Conventions de lecture
- 4.7 Recalibration & sélection
- 4.8 Les concepts du mécanisme
- 4.9 Protocole, code, reproductibilité

5. Limites & l'étape manquante

1. Le verdict — les trois régimes

1.1 Régime weekly-natif — NsDiff domine les deux axes

Les deux modèles sont entraînés sur rendements hebdomadaires, sur les mêmes 30 origines du duel, à 200 tirages. Le tableau donne RMSE (point) et verdict Winkler (incertitude) par horizon × actif. « graines sig. » = nombre de graines sur 5 où le test par cellule rejette à 5 %.

Horizon	Actif	RMSE NsDiff / TSDiff	Verdict RMSE	gr. sig.	Cov95 NsDiff / TSDiff	Verdict Winkler
W+1	BTC-USD	5255 / 5789	NsDiff	4/5	0.967 / 0.911	indistinguable
W+1	ETH-USD	274.0 / 349.4	NsDiff	4/5	0.947 / 0.927	NsDiff
W+1	SPY	12.95 / 13.19	indistinguable	0/5	0.944 / 0.802	NsDiff
W+1	TLT	1.402 / 1.790	NsDiff	5/5	0.964 / 0.956	NsDiff
W+1	ZN=F	0.7725 / 0.7712	indistinguable	0/5	0.933 / 0.827	NsDiff
W+2	BTC-USD	8118 / 9567	NsDiff	4/5	0.949 / 0.898	NsDiff
W+2	ETH-USD	413.6 / 632.8	NsDiff	4/5	0.962 / 0.884	NsDiff
W+2	SPY	17.07 / 17.61	indistinguable	0/5	0.951 / 0.811	NsDiff
W+2	TLT	1.832 / 2.820	NsDiff	4/5	0.976 / 0.924	NsDiff
W+2	ZN=F	0.9722 / 1.017	NsDiff	1/5	0.962 / 0.793	NsDiff
W+3	BTC-USD	10 610 / 12 760	NsDiff	4/5	0.942 / 0.891	NsDiff
W+3	ETH-USD	525.8 / 849.4	NsDiff	4/5	0.936 / 0.880	NsDiff
W+3	SPY	20.82 / 21.76	indistinguable	1/5	0.920 / 0.818	NsDiff
W+3	TLT	2.073 / 3.538	NsDiff	5/5	0.987 / 0.947	NsDiff
W+3	ZN=F	1.143 / 1.237	NsDiff	0/5	0.964 / 0.838	NsDiff

Poolé tous actifs : NsDiff significativement meilleur sur les 6 tests (3 horizons × RMSE/Winkler), $p = 0$ partout (zéro réplicat bootstrap sur 10 000 du mauvais côté — soit $p < 2 \times 10^{-4}$, cf. §4.5). TSDiff ne gagne aucune cellule sur aucun axe.

1.2 Régime daily — égalité sur le point, écart massif sur l'incertitude

Les deux modèles quotidiens sont poussés multi-step vers la cible-vendredi. Poolé tous actifs :

Horizon	Poolé RMSE	Poolé Winkler	Cov95 NsDiff	Cov95 TSDiff
W+1	indistinguable ($p=0.152$)	NsDiff ($p=0$)	0.882 – 0.933	0.598 – 0.738

Horizon	Poolé RMSE	Poolé Winkler	Cov95 NsDiff	Cov95 TSDiff
W+2	indistinguable (p=0.813)	NsDiff (p=0)	0.889 – 0.951	0.611 – 0.747
W+3	indistinguable (p=0.280)	NsDiff (p=0)	0.816 – 0.976	0.567 – 0.724

Sur le **point** (RMSE), les deux diffusions sont interchangeables en daily : aucun test poolé ne les sépare, et les verdicts par cellule sont majoritairement indistinguables. Sur l'**incertitude**, l'écart est sans exception : **20 à 30 points de couverture d'avance pour NsDiff**, et le Winkler en sa faveur sur les **15 cellules / 15**. TSDiff-D couvre 0,57–0,75 là où la cible est 0,95 — sous-calibré, ses intervalles sont trop étroits.

1.3 Régime mensuel — asymétrie entre les deux sous-régimes

À l'horizon mensuel (M+1/M+2/M+3), la comparaison se scinde nettement. Tableau M+1, RMSE testé par cellule (effective_n=13) :

Sous-régime	Actif	RMSE TSDiff	RMSE NsDiff	Cov95 TSDiff	Cov95 NsDiff	Verdict	p
C (mensuel)	BTC-USD	28 566	12 156	100%	100%	NsDiff	<2×10 ⁻⁴
C (mensuel)	ETH-USD	1132	755.2	100%	100%	NsDiff	0.0030
C (mensuel)	SPY	88.26	20.37	97.5%	100%	NsDiff	<2×10 ⁻⁴
C (mensuel)	TLT	10.60	3.315	95%	100%	NsDiff	<2×10 ⁻⁴
C (mensuel)	ZN=F	5.876	1.962	97.5%	100%	NsDiff	<2×10 ⁻⁴
B (daily)	BTC-USD	9281	9675	7.5%	95%	indist.	0.2424
B (daily)	ETH-USD	493.7	513.4	72.5%	95%	indist.	0.2402
B (daily)	SPY	19.89	20.55	30%	95%	indist.	0.6244
B (daily)	TLT	3.344	3.323	22.5%	90%	indist.	1.0000
B (daily)	ZN=F	1.843	2.005	25%	90%	TSDiff	0.0120

Attention à la lecture du sous-régime B (daily) mensuel. La couverture de TSDiff s'effondre à 7,5–30 % (cible 95 %) : c'est la signature d'un *collapse de diffusion* (§4.8), pas une comparaison de précision propre. La cellule « ZN=F, TSDiff meilleur, p=0.012 » et les 4 « indistinguable » ne doivent **pas** être lues comme un avantage de point — le Winkler très dégradé de TSDiff (403 vs 87,5 sur SPY malgré un RMSE comparable) montre que l'écart vient de l'incertitude effondrée. À révéfier avec le nuage d'échantillons complet avant toute conclusion.

En mensuel-natif (régime C), en revanche, **NsDiff écrase TSDiff : 5/5 cellules significatives, RMSE 1,4× à 4,3× meilleur, et parfaitement stable — 25/25 (5 actifs × 5 graines)**. C'est le résultat le plus robuste de tout le corpus mensuel. Un modèle plus petit généralise mieux sur seulement 34–70 fenêtres d'entraînement mensuelles qu'un gros UNet qui sur-apprend.

1.4 Bilan des 30 cellules weekly + daily (2 régimes × 3 horizons × 5 actifs)

Axe	NsDiff gagne	TSDiff gagne	indistinguable
Winkler (incertitude)	29	0	1

Axe	NsDiff gagne	TSDiff gagne	indistinguable
RMSE (point)	12	2	16

Le motif est net : le gain de NsDiff se loge dans la fiabilité de l'incertitude, pas dans le point. En weekly il gagne les deux ; en daily, seulement la calibration.

2. Le mécanisme — pourquoi NsDiff bat TSDiff en weekly mais l'égale en daily

Ce n'est pas une conjecture : les chiffres désignent une cause unique — le **rapport paramètres / données**.

Le fait à expliquer

Surcroît de RMSE de TSDiff par rapport à NsDiff, en % (positif = TSDiff moins précis), moyenne sur les 5 actifs :

Régime	W+1	W+2	W+3
weekly	+13,4 %	+26,5 %	+33,0 %
daily	-0,8 %	+0,5 %	+2,5 %

En weekly, l'écart **se creuse le long du chemin généré**. En daily, il est plat et quasi nul.

Pourquoi RMSE ici, et pas CRPS ? Parce que ce chapitre *diagnostique une panne précise*, il ne classe pas les modèles. Le CRPS est bien le score de référence global d'une prévision probabiliste, mais il **agrège** justesse du centre *et* honnêteté de la dispersion en un seul chiffre. Or le mécanisme démontré ici est un mécanisme *de moyenne* : la moyenne conditionnelle de TSDiff est mal estimée, et comme W+2/W+3 se lisent sur la somme cumulée, ce biais *de centre* se compose (+13 % → +27 % → +33 %). La **RMSE isole exactement le point** — elle mesure la dérive du centre et rien d'autre. Un CRPS à ces mêmes horizons serait *pollué* par l'effondrement simultané de la dispersion de TSDiff (le collapse, §4.8) : on ne saurait plus si la divergence vient de la moyenne ou de l'incertitude — soit précisément la distinction que tout le document construit. L'axe probabiliste n'est pas oublié pour autant : il est porté séparément par le Winkler, la Cov95 et le CRPS (là où le nuage complet est conservé), et expliqué par le corollaire « backbone de sigma ». Deux axes complémentaires, tenus séparés — c'est la décomposition « le point vs l'incertitude ». (Deux précisions : pour une prévision réduite à un point, le CRPS se ramène à l'erreur absolue — une norme L1 — pas à la RMSE, qui est L2 : ce ne sont même pas la même famille. Et le CRPS exige le nuage complet des tirages, conservé seulement en weekly, tandis que la RMSE est calculable dans les trois régimes — le dénominateur commun d'un argument qui les traverse tous.)

La cause : le rapport paramètres / données

Modèle	Régime	Paramètres	Fenêtres d'entraînement	Paramètres / fenêtre
TSDiff	weekly	251 111	478	525
NsDiff	weekly	8 360	478	17
TSDiff	daily	251 135	2 420	104
NsDiff	daily	9 152	2 420	4

TSDiff est ~30× plus gros que NsDiff. En weekly, il doit estimer 251 000 paramètres avec 478 fenêtres : sur-paramétrisation sévère, sa moyenne conditionnelle est mal estimée, et comme les horizons W+2/W+3 se lisent sur

la *somme cumulée* du chemin échantillonné, ce biais par pas **se compose** (d'où la divergence monotone +13 % → +27 % → +33 %). En daily, TSDiff passe à 104 paramètres par fenêtre — 5× de mieux — et **l'écart disparaît intégralement**.

Le test qui exclut l'explication concurrente. L'hypothèse rivale serait « c'est la longueur du chemin qui pénalise ». Elle est *réfutée par les données* : en daily le modèle traverse **5× plus de pas de diffusion** pour atteindre les mêmes cibles (5/10/15 pas au lieu de 1/2/3), et pourtant il compose *moins* d'erreur (+2,5 % à W+3 contre +33 %). Si la longueur du chemin était le mécanisme, le classement serait inversé. Ce qui varie dans le bon sens, c'est le volume de données.

Trois corollaires, tous du même mécanisme

(1) Pourquoi TSDiff-D s'effondre au sur-entraînement. 251 000 paramètres sur 2 420 fenêtres, c'est encore assez pour mémoriser : au-delà de ~20 époques, la variance résiduelle tend vers zéro et l'échantillonneur cesse de produire une loi — largeur du PI de 12,4 % → 0,12 % du prix entre 10 et 80 époques. NsDiff, à 4 paramètres/fenêtre, ne peut structurellement pas mémoriser.

(2) Pourquoi NsDiff domine toujours la calibration (29 cellules/30 au Winkler). Son *backbone de sigma* modélise **explicitement** la volatilité. Chez TSDiff, la dispersion est un sous-produit du processus de bruit, sans paramétrisation dédiée — c'est donc la première chose que le sur-apprentissage détruit.

(3) Pourquoi le match daily est nul sur le point mais pas sur l'incertitude. Avec assez de données, TSDiff rattrape la moyenne conditionnelle ; il ne rattrape pas la dispersion, parce que rien dans son architecture ne la modélise.

Réserve à ne pas gommer. Le budget d'époques est un confondant partiel du match nul en daily. À son budget d'origine (80 époques, calibration détruite), TSDiff-D avait un RMSE *meilleur* de 0,2 à 3,8 % qu'à 20 (SPY 12,99 @80ép vs 13,49 @20ép ; NsDiff-D 13,26). Autrement dit : à son budget optimal en RMSE, TSDiff-D devancerait légèrement NsDiff-D *sur le point* — mais avec 1 à 6 % de couverture. L'« égalité » du §1.2 vaut au budget sélectionné en CRPS, le seul où les deux modèles produisent tous deux une distribution exploitable.

2.1 Pré-training, capacité, biais inductif — d'où vient l'écart (et d'où il ne vient PAS)

Le tableau paramètres/données ci-dessus *constate* l'écart de capacité ; il ne dit pas *pourquoi* cet écart existe ni ce qui, dans l'entraînement des deux modèles, en découle. En creusant le code des modèles (`models/tsdiff_model.py` , `models/nsdiff_model.py`), on peut séparer proprement trois choses qu'on confond souvent : le **pré-training**, la **capacité** (nombre de paramètres) et le **biais inductif** (les choix de design, comme la structure sigma). La conclusion recadre la question du tuteur.

Ce qui est identique entre les deux — donc PAS la cause (vérifié dans le code)

Dimension d'entraînement	TSDiff	NsDiff	→ cause de l'écart ?
Pré-training / transfert	aucun — <i>from scratch</i>	aucun — <i>from scratch</i>	non
Protocole	train-once-forward, par actif	train-once-forward, par actif	non
Optimiseur / LR	AdamW, 2×10^{-4}	AdamW, 2×10^{-4}	non
Époques (défaut)	40	40	non

Dimension d'entraînement	TSDiff	NsDiff	→ cause de l'écart ?
seq_len / k_denoise / n_samples	≤30 / 20 / 50	≤30 / 20 / 50	non
Données / fenêtres	mêmes prix gelés	mêmes prix gelés	non

Aucun des deux n'est pré-entraîné — le dépôt ne charge aucun poids pré-appris (pas de `load_state_dict` depuis disque, pas de `from_pretrained`, pas de `torch.load` de checkpoint). Les deux sont ajustés *from scratch* sur chaque actif, avec la **même recette d'entraînement exacte** (mêmes optimiseur, LR, époques, budgets). C'est décisif : l'écart n'est **ni du pré-training, ni un artefact d'hyperparamètres, ni une question de données**. Ça ne laisse que deux causes possibles — la capacité et le biais inductif — et elles sont liées.

Différence 1 — la capacité (le nombre de paramètres, et le chiffre à corriger)

Le « 8 à 9000 paramètres », c'est **NsDiff**, pas TSDiff. TSDiff en compte **~251 000**, soit **~30×** plus. L'origine est architecturale : TSDiff est un *UNet1D* (canaux cachés 64, profondeur 2) empilant blocs résiduels, attention de canal, décomposition apprise, conditionnement FiLM et embedding sinusoïdal (`tsdiff_model.py:69-77`). NsDiff est trois petits MLP (largeur 32) : moyenne, sigma, débruiteur. Fait notable du port (`nsdiff_model.py` , en-tête) : *même l'officiel NsDiff est « enormously over-parameterised for this benchmark »* (son backbone de moyenne est un Transformer non-stationnaire, `d_model=512`, 8 têtes) — il a donc été **délibérément dégraissé** ici (moyenne remplacée par un MLP 2 couches, largeurs 512 → quelques dizaines), en *préservant* le cœur scientifique (le backbone de sigma `g_psi` et le débruiteur). Autrement dit, la petitesse de NsDiff n'est pas un hasard : c'est un choix, et il est permis par le point suivant.

Différence 2 — le biais inductif (pourquoi la structure sigma n'est pas « qu'un » choix de design)

La différence de fond entre les deux tient en deux équations, transcrites du code :

TSDiff : $Y = f(X) + \varepsilon$ (bruit à variance **constante**)
NsDiff : $Y = f(X) + \sqrt{g(X)} \cdot \varepsilon$ (modèle **location-scale**, $g(X)$ = variance conditionnelle)

NsDiff *injecte* la volatilité comme structure — `g_psi` est un réseau de variance conditionnelle, décrit dans le code comme « **the generative analogue of GARCH** ». TSDiff, lui, ajoute un bruit de largeur fixe : sa dispersion doit *émerger* du processus de diffusion, avec de la capacité générique et aucun organe dédié. Le tuteur a raison — la structure sigma *est* un choix de design mathématique. Mais c'est un **biais inductif fort**, et un biais inductif **remplace de la capacité et des données** : c'est précisément ce choix qui *permet* à NsDiff d'être 30× plus petit tout en calibrant mieux.

Le point clé. « Choix de design » et « capacité / entraînement » ne sont pas deux compartiments séparés : **le choix de design EST le mécanisme de capacité**. En modélisant la variance explicitement, NsDiff n'a pas à *apprendre* la dispersion depuis les données — donc il lui faut moins de paramètres, donc il généralise sur peu de fenêtres, donc il ne s'effondre pas. Chez TSDiff, la dispersion est un sous-produit non paramétré : c'est la première chose que le sur-apprentissage détruit. La « structure sigma » n'est donc pas un détail cosmétique à côté du mécanisme paramètres/données — elle en est *la source*.

La chaîne de conséquences

Une fois posé qu'il n'y a pas de pré-training, l'enchaînement est mécanique. **Côté TSDiff** : pas de pré-training → sa capacité ne s'amortit sur aucun corpus → 251 000 paramètres à apprendre depuis les seules ~478 fenêtres weekly d'un actif → sur-paramétrisation → sur-apprentissage → collapse de l'échantillonneur (variance résiduelle → 0, largeur du PI 12,4 % → 0,12 % entre 10 et 80 époques) → couverture effondrée. **Côté NsDiff** : prior structurel + 8-

9k paramètres → efficient en données → stable (pas de collapse) → calibration bonne par construction (`g_psi` dédié). Et une conséquence d'entraînement mesurable : la calibration de TSDiff varie d'un **facteur 100** entre 10 et 80 époques, quand NsDiff reste stable à 40 — la fragilité vient de la capacité excédentaire, pas d'un mauvais réglage (le réglage est identique).

L'angle pré-training, justement — et pourquoi le verdict est conditionnel. Ici, personne n'est pré-entraîné *cross-actifs*. Or c'est exactement là qu'un gros modèle justifie normalement sa capacité : un diffuseur de séries temporelles *peut* être pré-entraîné sur un grand corpus de séries (logique « foundation model ») puis transféré/guidé à l'inférence — auquel cas les 251 000 paramètres de TSDiff deviendraient un *atout* (capacité amortie sur beaucoup de données) au lieu d'un handicap (capacité à apprendre depuis un seul actif). Ce benchmark **ne fait pas ça** — train-once-forward, par actif, from scratch. Donc le verdict « le petit modèle bat le gros » est vrai **dans ce régime précis, sans pré-training** ; il pourrait s'inverser sous un pré-training cross-séries. C'est une expérience distincte, non faite, à déclarer comme telle — et c'est sans doute la piste la plus intéressante à explorer ensuite sur cet axe.

3. Deux erreurs de protocole rencontrées en route

3.1 Le premier bras daily était invalide (corrigé)

La première génération réutilisait, côté daily, le budget d'époques hebdomadaire de chaque modèle. C'est **sûr pour NsDiff** (40 plat) mais **faux pour TSDiff** : la série quotidienne compte ~2465 observations contre ~511 en hebdomadaire, donc à époques égales ~5× plus de pas de gradient — donc un TSDiff quotidien sur-entraîné, dont la couverture tombait à 1–6 %. Correction : `generate_nsdiff_asset` accepte désormais un `epochs_daily` distinct (opt-in), et le budget de TSDiff-D est sélectionné par validation (argmin CRPS, candidats 10/20/40). Retenu : 20 époques dans 19 cellules sur 25. **Vérifié bit-à-bit** : le bras weekly se reproduit à l'identique après correction.

3.2 Une fuite dans la sélection d'époques hebdomadaire de TSDiff (déclarée, non corrigée)

Les époques hebdomadaires de TSDiff (40/60/80) ont été sélectionnées sur une validation dont les 12 origines tombent *dans* la grille de test `oos`. ~13 % des origines évaluées ont donc servi à choisir l'hyperparamètre de TSDiff. **Non corrigé — mais la fuite joue en faveur de TSDiff**, qui perd quand même les 6 tests poolés weekly. Le verdict n'en est que plus conservateur. Le sweep daily du §3.1, lui, est ancré strictement avant la première origine de test.

4. Glossaire étendu

Chaque entrée : **définition** (ce que c'est), **explication** (pourquoi ça compte ici), **exemple** (un cas concret du duel ou du code). Les renvois `fichier.py:ligne` pointent le code réel du dépôt qui produit la notion.

4.1 Les modèles, l'architecture et la diffusion

Modèle génératif (par diffusion)

DÉFINITION. Un modèle qui, au lieu de renvoyer directement une fourchette, apprend à générer des *échantillons* de trajectoires de prix futurs en partant de bruit pur et en le « débruitant » pas à pas. La fourchette est ensuite lue comme des quantiles de ces échantillons.

EXPLICATION. C'est ce qui rend le « budget de tirages » (§4.3) central : la qualité des quantiles dépend directement du nombre d'échantillons générés. Deux modèles génératifs ne sont comparables que s'ils tirent le même nombre de trajectoires.

EXEMPLE. Pour prévoir SPY à 3 semaines, NsDiff tire 200 trajectoires ; la borne haute à 95 % est le 97,5^e percentile empirique de ces 200 valeurs à l'horizon visé.

Processus de diffusion & débruitage (k_{denoise} , pas de diffusion)

DÉFINITION. Le mécanisme génératif en deux temps. *Forward* : on ajoute progressivement du bruit à une trajectoire réelle jusqu'à obtenir du bruit pur. *Reverse (débruitage)* : le réseau apprend à inverser chaque pas — partir de bruit et le nettoyer graduellement pour produire une trajectoire plausible. Le nombre de pas de nettoyage est k_{denoise} (20 chez TSDiff).

EXPLICATION. Chaque « pas de diffusion » est une étape de ce nettoyage. Plus il y a de pas, plus l'échantillon est raffiné mais plus c'est coûteux. C'est un levier distinct de la *longueur du chemin temporel* prévu (§4.8) : on peut avoir beaucoup de pas de diffusion pour un horizon court.

EXEMPLE. En daily, un modèle traverse 5× plus de pas *temporels* qu'en weekly pour viser la même cible (5/10/15 au lieu de 1/2/3) — à ne pas confondre avec k_{denoise} , qui, lui, ne change pas.

NsDiff (+ débruiteur conditionnel)

DÉFINITION. Modèle génératif **maigre** (~8 000–9 000 paramètres) : un empilement de trois petites briques — un *backbone de moyenne* (petit MLP), un *backbone de sigma* (volatilité), et un petit *débruiteur conditionnel* (le réseau qui exécute le reverse, conditionné sur l'historique).

EXPLICATION. Le point clé est le backbone de sigma : NsDiff modélise la **volatilité explicitement**. C'est structurellement pourquoi il calibre mieux et pourquoi il ne peut pas mémoriser (4 paramètres/fenêtre en daily). « Conditionnel » = le débruitage dépend du contexte passé (les rendements récents), pas seulement du bruit.

EXEMPLE. Le fit d'un actif prend quelques secondes, là où TSDiff prend 176–409 s par (actif, graine) sur le run mensuel.

TSDiff

DÉFINITION. Modèle génératif **lourd** (~251 000 paramètres), un *UNet1D de diffusion* (voir entrée suivante).

EXPLICATION. ~30× plus gros que NsDiff. Atout quand les données abondent (daily : 2 420 fenêtres), handicap quand elles manquent (weekly : 478 ; mensuel : 34–70) : il sur-apprend, et sa dispersion — non paramétrée — s'effondre.

EXEMPLE. En weekly, il doit estimer 525 paramètres par fenêtre contre 17 pour NsDiff ; d'où la divergence de RMSE qui se creuse (+13 % → +33 %).

Architecture UNet1D : blocs résiduels, attention de canal, décomposition apprise, embedding sinusoïdal

DÉFINITION. Les briques internes de TSDiff. *UNet1D* : réseau en « U » (compression puis reconstruction) adapté aux séries 1D. *Blocs résiduels* : couches avec raccourci qui ajoute l'entrée à la sortie (facilite l'entraînement profond). *Attention de canal* : mécanisme qui pondère l'importance relative des différents canaux de features. *Décomposition apprise* : séparation tendance/saisonnalité *apprise* par le réseau plutôt que fixée. *Embedding sinusoïdal* : encodage du numéro de pas de diffusion en vecteurs de sinus/cosinus (comme les Transformers encodent la position).

EXPLICATION. Ces briques donnent à TSDiff sa capacité — et son poids (251 k paramètres). C'est cette richesse qui devient un handicap en petit régime : beaucoup de paramètres à estimer avec peu de fenêtres.

EXEMPLE. NsDiff n'a rien de tout ça : pas d'attention, pas de UNet — juste deux petits MLP (moyenne, sigma) et un débruiteur. La différence de calibration vient précisément de là.

Backbone (d'un réseau de neurones)

DÉFINITION. Dans un réseau de neurones, le *backbone* (« colonne vertébrale ») est le sous-réseau principal qui fait le gros du travail : transformer l'entrée (l'historique des rendements) en une représentation utile, à partir de laquelle on lit la quantité voulue.

EXPLICATION. C'est un terme d'architecture : un modèle peut avoir un seul backbone (tout passe par lui) ou plusieurs, chacun spécialisé sur une sortie. NsDiff en a deux, dédiés — un pour la tendance, un pour la volatilité. C'est ce découpage explicite qui fait sa force en calibration.

EXEMPLE. Chez TSDiff, il n'y a qu'un gros backbone (le UNet1D) d'où sont lues à la fois moyenne et dispersion ; la dispersion n'a pas d'organe à elle, elle est un sous-produit — d'où sa fragilité.

Backbone de moyenne / backbone de sigma

DÉFINITION. Les deux sous-réseaux de NsDiff qui prédisent respectivement la *tendance* (moyenne conditionnelle) et la *dispersion* (sigma, volatilité conditionnelle) de la loi de prix future.

EXPLICATION. Séparer les deux permet à NsDiff de garder une bonne dispersion même quand le point est difficile ; TSDiff n'a pas d'organe dédié à la dispersion, qui n'est chez lui qu'un sous-produit du bruit de diffusion.

EXEMPLE. En daily, les deux modèles sont à égalité sur le point mais NsDiff garde $\text{Cov95} \approx 0,88\text{--}0,95$ contre $0,57\text{--}0,75$ pour TSDiff : c'est le backbone de sigma qui fait la différence.

DiffusionEngine · engine, ddim_eta, échantillonnage ancestral / DDIM

DÉFINITION. Un *engine* (moteur) est l'objet logiciel qui emballe « quel modèle » + « comment il génère ses tirages » derrière une seule interface. `generate_nsdiff_asset(...)` reçoit un `DiffusionEngine` via le kwarg `engine` : changer d'engine change de modèle (NsDiff ou TSDiff) et de procédure d'échantillonnage, *sans toucher au reste du code* autour (fit, standardisation, lecture des quantiles restent les mêmes).

EXPLICATION. C'est ce qui rend le duel *propre* : exactement la même tuyauterie sert les deux modèles, seul l'engine diffère. Les réglages d'échantillonnage sont propres à TSDiff — *échantillonnage ancestral* (rejouer tout le débruitage stochastique, pas par pas) vs *DDIM* (variante accélérée, dont `ddim_eta` règle la stochasticité, 1.0 = équivalent ancestral complet ; « nécessaire pour un vrai PI », dit `tsdiff_model.py`). NsDiff fait de l'ancestral complet, sans DDIM.

EXEMPLE. Le chemin par défaut (NsDiff) a été vérifié *bit-à-bit* équivalent à l'appel d'origine sur fit, standardisation et forecast.

ARIMA-GARCH / GARCH(1,1) — le modèle de volatilité de référence

DÉFINITION. Le benchmark classique. *ARIMA* modélise la moyenne (tendance/autocorrélation) ; *GARCH(1,1)* modélise la **variance conditionnelle** — la volatilité d'aujourd'hui dépend du choc et de la volatilité d'hier. C'est le mur à battre : aucun modèle de diffusion ne le dépasse sous SPA.

EXPLICATION. NsDiff a été choisi précisément parce que son backbone de sigma est l'*analogue diffusion* de GARCH (volatilité conditionnelle explicite). La comparaison NsDiff-vs-GARCH est donc la plus significative — mais asymétrique : GARCH est refit à chaque origine, NsDiff est train-once-forward (§4.2).

EXEMPLE. Duel CRPS : NsDiff gagne 1/5 vs ARIMA-GARCH aux trois horizons ; rejet SPA vs GARCH(1,1) toujours à 0/75. Le mur GARCH tient — d'où l'enjeu du backtest économique (§0).

4.2 Régimes, horizons, origines, données

Régime daily (B) — « daily-poussé »

DÉFINITION. Un modèle entraîné sur données *quotidiennes*, puis poussé multi-step jusqu'à la cible lointaine (vendredi de la semaine $W+k$, ou fin de mois $M+k$). Dans la DB : `frequency='daily'`, `horizon_type='weekly'` (ou `'monthly'`).

EXPLICATION. La prévision est lue à la **distance en jours de bourse réelle** jusqu'à la cible (jamais « +7 jours calendaires » en dur) : 5/10/15 pas quotidiens pour 1/2/3 semaines. Le régime B a beaucoup plus de fenêtres d'entraînement que le weekly (2 420 vs 478), ce qui neutralise la sur-paramétrisation de TSDiff.

EXEMPLE. C'est le régime où les deux modèles s'égalisent sur le point — parce que TSDiff a enfin assez de données (§2).

Régime weekly-natif (C)

DÉFINITION. Un modèle entraîné directement sur rendements *hebdomadaires* (rééchantillonnage `W-FRI + dropna()`). Dans la DB : `frequence='weekly'`.

EXPLICATION. Beaucoup moins de fenêtres (478 sur SPY), donc le régime qui punit le plus la taille de TSDiff : NsDiff y domine les deux axes.

EXEMPLE. Un modèle n'apparaît sur le dashboard que s'il a les *deux* côtés (daily-B et weekly-C) en `source='oos'` sur exactement les mêmes origines — c'est par construction une comparaison Daily-vs-Weekly par modèle.

Régime mensuel-natif

DÉFINITION. Même idée que le weekly-natif, mais sur rendements *mensuels* (cibles M+1/M+2/M+3). `frequence='monthly'`.

EXPLICATION. C'est le régime le plus pauvre en données (34–70 fenêtres d'entraînement), donc celui qui punit le plus un gros modèle. C'est là que NsDiff écrase le plus nettement TSDiff — 5/5 cellules, 25/25 graines.

EXEMPLE. RMSE mensuel-natif BTC-USD : NsDiff 12 156 vs TSDiff 28 566 (2,3× meilleur). Le petit modèle généralise, le gros sur-apprend.

Origine · cutoff_date · target_date · unité d'inférence

DÉFINITION. Une *origine* est un point de départ de prévision : « je me place à telle date, je ne connais que le passé, et je prévois telle date future ». Elle est décrite par un triplet (`asset`, `cutoff_date`, `target_date`) — `cutoff_date` = la dernière date connue (là où je m'arrête) ; `target_date` = la date que je cherche à prévoir.

EXPLICATION. C'est la brique de base de toute l'évaluation. Le *walk-forward* fait glisser cette origine dans le temps : origine 1, puis origine 2 une semaine plus loin, etc. — 30 origines par actif, 90 en tout (les 3 horizons). On appelle l'origine l'**unité d'inférence** parce que c'est *l'objet qu'on compte* (n=90) et *qu'on rééchantillonne* dans le bootstrap (§4.5). Moyenner sur 5 graines ne crée *aucune* nouvelle unité : on a toujours 90 origines, juste moins bruitées.

EXEMPLE CONCRET. Pour BTC-USD : `cutoff_date` = vendredi 5 juin 2026 (je ne connais les prix que jusque-là), `target_date` = vendredi 12 juin (le vendredi suivant, donc horizon W+1). Ce triplet est *une* origine. La semaine d'après, la même chose décalée d'une semaine = l'origine suivante. Pour que le duel soit équitable, NsDiff et TSDiff doivent tomber sur exactement les mêmes triplets — d'où la lecture *verbatim* des origines des baselines déjà en base (`load_baseline_triplets`), jamais régénérées « à peu près ».

Walk-forward & point-in-time (anti-fuite)

DÉFINITION. *Walk-forward* : on évalue en avançant origine par origine dans le temps, comme on le ferait en vrai — à chaque date, on prévoit puis on avance. *Point-in-time* : à chaque origine, le modèle ne voit **que** l'information réellement disponible à cette date ; rien du futur ne « fuit » dans la prévision (pas de *lookahead*).

EXPLICATION. Une *fuite* (*lookahead*), c'est utiliser sans le vouloir une donnée postérieure au `cutoff_date`. Exemple typique à éviter : standardiser (§Standardisation) avec une moyenne `mu` calculée sur *toute* la série, y compris les mois qui suivent l'origine — le modèle « saurait » alors quelque chose du futur, et l'évaluation serait faussement bonne. Le point-in-time interdit ça : `mu / sd` sont gelés sur le seul passé, et la fenêtre d'historique grandit mais ne dépasse jamais l'origine (`weekly_z[:m]` s'arrête à l'origine `m`). C'est ce qui rend un backtest honnête plutôt qu'illusoire.

EXEMPLE. La recalibration conforme (§4.7) applique la même règle : elle découpe calibration/test par une `cutoff_date` unique, la calibration étant *strictement avant* le test — jamais l'inverse, sinon on calibrerait sur des données qu'on est censé prédire.

Train-once-forward vs refit périodique

DÉFINITION. *Train-once-forward* : le modèle est entraîné **une seule fois**, sur les données antérieures à la première origine, puis appliqué à toutes les origines suivantes sans ré-entraînement. *Refit périodique* : ré-entraîne à intervalles (ex. GARCH, à chaque origine).

EXPLICATION. NsDiff et TSDiff sont tous deux train-once-forward dans le duel, donc **symétriques** — comparaison propre. Ce n'est PAS le cas face à GARCH (refit à chaque origine) : là, l'écart mesuré mélange « modèle » et « protocole d'entraînement », réserve à citer.

EXEMPLE. En production, la cadence de refit de NsDiff resterait à définir — un des écarts opérationnels de la big picture (§0).

Horizon (W+1/W+2/W+3, M+1/M+2/M+3) & somme cumulée

DÉFINITION. Le nombre de périodes projetées. $W+k$ = k semaines ; $M+k$ = k mois. La cible à l'horizon k se lit sur la **somme cumulée** du chemin échantillonné (prix à k périodes = départ + somme des pas générés).

EXPLICATION. C'est pourquoi un biais par pas *se compose* avec l'horizon : si la moyenne conditionnelle est biaisée à chaque pas, l'erreur s'accumule à $W+2$ puis $W+3$.

EXEMPLE. TSDiff weekly : +13,4 % ($W+1$) → +26,5 % ($W+2$) → +33,0 % ($W+3$). La pente est la signature de la composition.

Standardisation (`mu / sd`, z-scores)

DÉFINITION. Avant l'entraînement, les rendements sont centrés-réduits : $z = (x - \mu) / sd$, où μ (moyenne) et sd (écart-type) sont calculés sur le train seul. Le modèle travaille en z-scores ; on inverse la transformation pour repasser en prix.

EXPLICATION. Geler `mu / sd` au train est une condition *point-in-time* : les recalculer avec des données futures serait une fuite. C'est aussi ce qui met tous les actifs à la même échelle pour le réseau.

EXEMPLE. `weekly_z[:m]` = les z-scores hebdomadaires jusqu'à l'origine `m`, jamais au-delà.

Fit (ajuster / entraîner) & approche par actif (univariée)

DÉFINITION. « Fitter » un modèle, c'est l'*entraîner* sur des données : régler ses paramètres internes pour qu'il reproduise au mieux le comportement observé. Le « fit d'un actif » = un modèle entraîné **séparément pour chaque actif** (BTC-USD, SPY, TLT...), sur son seul historique — approche dite *univariée* / par actif, par opposition à un modèle global multi-actifs.

EXPLICATION. Chaque actif a son propre modèle fitté, avec ses propres `mu / sd` et ses propres paramètres. Le coût d'un fit dépend de la taille du modèle et du nombre de pas de gradient ; c'est là que se voit l'écart NsDiff (léger) / TSDiff (lourd).

EXEMPLE. Le fit d'un actif prend quelques secondes pour NsDiff (deux petits MLP) contre 176–409 s pour TSDiff (UNet + 1000 pas de diffusion à l'entraînement) — d'où un run 5 graines mensuel de ~152 min côté TSDiff.

Époques, `seq_len`, pas de gradient

DÉFINITION. *Époque* : un passage complet sur les données d'entraînement. *Pas de gradient* : une mise à jour des poids (il y en a plusieurs par époque, un par batch). `seq_len` : la longueur de la fenêtre d'historique donnée en entrée (30 ici).

EXPLICATION. À nombre d'époques égal, une série plus longue = plus de pas de gradient = plus d'entraînement effectif. C'est l'*erreur du §3.1* : réutiliser le budget d'époques weekly côté daily donne ~5× plus de pas de gradient à TSDiff-D (série quotidienne ~5× plus longue), donc sur-entraînement.

EXEMPLE. Budgets déclarés du duel : `epochs=40` , `seq_len=30` , `k_denoise=20` .

Appariement daily/weekly (`build_daily_weekly_pairs`)

DÉFINITION. L'opération qui, pour chaque modèle, apparie sa prévision côté daily (régime B) et côté weekly (régime C) qui partagent le *même* `target_date` , afin de comparer les deux régimes sur exactement les mêmes cibles.

EXPLICATION. Sans cet appariement à cible identique, on comparerait des choux et des carottes. C'est aussi pourquoi les origines doivent être partagées : pas d'appariement possible sinon.

EXEMPLE. Le dashboard attend « 30 paires = 6 modèles × 5 actifs » à W+1 ; un modèle sans ses deux côtés en `oos` disparaît de la comparaison.

4.3 Le budget d'échantillonnage, les intervalles et les quantiles

Budget (`n_samples`, `m`) — « budget d'échantillonnage égal »

DÉFINITION. Le nombre de tirages (trajectoires) générés par prévision — le `m` du nuage. `n_samples` dans le code, un simple paramètre de `generate_nsdiff_asset` (défaut 50, porté à 200 pour le duel).

EXPLICATION. La qualité des quantiles dépend directement de `m`. Comparer un modèle à 200 tirages contre un autre à 50 offrirait un avantage *au protocole*, pas gagné par le modèle. D'où les non-négociables : 200 des deux côtés, mêmes prix gelés, même formule, même boucle.

EXEMPLE. C'est comme comparer deux archers en donnant le même nombre de flèches à chacun : sinon on mesure le carquois, pas l'archer.

Intervalle de prédiction (PI) & niveau (95 %, α)

DÉFINITION. Le *PI* (Prediction Interval) est la fourchette [borne basse, borne haute] censée contenir la réalisation avec une probabilité donnée. Le *niveau* est cette probabilité (95 % ici) ; $\alpha = 1 - \text{niveau} = 0,05$.

EXPLICATION. Un PI à 95 % se lit comme les quantiles empiriques 2,5 % et 97,5 % du nuage. La *largeur* du PI mesure la confiance du modèle ; sa *couverture* mesure si cette confiance est justifiée.

EXEMPLE. Le collapse de TSDiff se lit sur la largeur du PI en % du prix : 12,4 % à 10 époques → 0,12 % à 80 — un intervalle devenu absurdement étroit.

Quantile empirique & statistique d'ordre

DÉFINITION. Une *statistique d'ordre* est une valeur d'un échantillon une fois trié (la 1^{re} = le minimum, la n-ième = le maximum). Un *quantile empirique* au niveau q est la statistique d'ordre correspondant à la position q dans l'échantillon trié.

EXPLICATION. Avec peu de tirages, il n'y a pas de statistique d'ordre « pile » au bon endroit : on tombe sur la plus proche, qui sous-estime la vraie queue. C'est la mécanique exacte du biais du quantile à faible m .

EXEMPLE. Sur 50 tirages, le quantile 97,5 % est la ~49^e statistique d'ordre sur 50 ($49/50 = 0,98$) : elle est *en deçà* de la vraie borne 97,5 %, donc l'intervalle couvre moins que 95 %.

Biais du quantile empirique (à faible m)

DÉFINITION. À petit m , le quantile empirique 97,5 % est *biaisé vers l'intérieur* : l'intervalle « étiqueté 95 % » couvre en réalité moins.

EXPLICATION. À 50 tirages, un intervalle « 95 % » ne couvre réellement que ~91,3 %. Bandes trop étroites → trop de violations → Winkler pénalisé — un déficit de couverture *artefactuel*, dû au budget, pas au modèle.

EXEMPLE. Une partie du « signal réel de calibration en faveur du weekly » d'une synthèse antérieure était en fait ce biais à 50 tirages ; testé proprement à 200, il se réduit fortement (~2,8 points de couverture rendus à NsDiff).

4.4 Les métriques

RMSE (Root Mean Squared Error)

DÉFINITION. Racine de la moyenne des erreurs de prévision au carré sur le *point* (valeur centrale prédite vs réalisée). Plus petit = meilleur.

EXPLICATION. Métrique de **point** : elle ne dit rien de l'intervalle. C'est l'axe où NsDiff et TSDiff s'égalisent en daily.

EXEMPLE. Weekly W+1 ETH-USD : RMSE NsDiff 274,0 vs TSDiff 349,4 → NsDiff meilleur sur le point.

Cov95 (couverture empirique à 95 %)

DÉFINITION. Fraction des réalisations effectivement tombées dans le PI à 95 %. Cible : 0,95.

EXPLICATION. $< 0,95$ = intervalles trop étroits (sous-couverture, sur-confiance) ; $> 0,95$ = trop larges (sur-couverture, prudence excessive). Code : `in_interval = (y_true ≥ y_lower) & (y_true ≤ y_upper)`, moyennée.

EXEMPLE. Daily W+1 : NsDiff couvre 0,88–0,93 (proche cible) contre TSDiff 0,60–0,74 (largement sous-calibré).

Winkler / Interval Score

DÉFINITION. Règle de scoring propre pour un intervalle (Gneiting & Raftery 2007) : la **largeur** de l'intervalle, *plus* une pénalité $(2/\alpha) \times \text{dépassement}$ si la réalisation tombe en dehors. Plus petit = meilleur. Vérifié `dashboard_d7_w1.py:74`.

EXPLICATION. Seule métrique probabiliste calculable quand la DB ne stocke que `(y_lower, y_upper)`. Elle capture d'un chiffre le compromis *largeur* $\times$ *couverture* : étroit mais qui rate = lourdement puni ; large et juste = paie seulement sa largeur.

EXEMPLE (SELF-TEST DU CODE). `lower=90, upper=110` (largeur 20), `$\alpha=0.05$` donc $2/\alpha=40$: réalisation à 100 (dedans) $\rightarrow 20$; à 115 (5 au-dessus) $\rightarrow 20+40 \times 5 = 220$; à 80 (10 en dessous) $\rightarrow 20+40 \times 10 = 420$.
(`_selftest_winkler`.)

CRPS empirique & CRPS « fair »

DÉFINITION. Le CRPS (Continuous Ranked Probability Score) note une prévision *distributionnelle* complète contre la réalisation. Sur un nuage (Gneiting & Raftery 2007, eq. 20) :

$$\text{CRPS}(F, y) \approx \text{moyenne}_i |x_i - y| - \frac{1}{2} \cdot \text{moyenne}_{i,j} |x_i - x_j|$$

EXPLICATION. 1^{er} terme = justesse (distance des tirages à la réalité) ; 2^e terme = dispersion (étalement du nuage). Plus fin que le Winkler, mais exige tout le nuage. Le `crps_fair` (Ferro et al. 2014) corrige un biais $O(1/m)$ du terme de dispersion à m fini, indispensable quand m diffère entre modèles. Code : `crps_metrics.py`.

EXEMPLE. Le CRPS n'est calculé que là où le nuage complet est conservé (protocole weekly). En mensuel et sur le dashboard, seuls point+quantiles sont stockés : pas de CRPS.

PIT (Probability Integral Transform)

DÉFINITION. Diagnostic de calibration : pour chaque réalisation, le rang qu'elle occupe dans la distribution prédite. Bien calibré $\rightarrow$ ces valeurs sont uniformes sur $[0,1]$.

EXPLICATION. Comme le CRPS, il a besoin du nuage complet ; disponible seulement dans le protocole weekly.

EXEMPLE. Un PIT « en U » (trop de réalisations aux extrêmes) signalerait des intervalles trop étroits — le symptôme de TSDiff sous-calibré.

Direction & Skill score vs Random Walk (RW)

DÉFINITION. *Direction* : le signe de la variation prédite (hausse/baisse) coïncide-t-il avec le réalisé (`direction_correct`). *Skill score* : performance *relative* à une baseline naïve — la marche aléatoire (point = dernier close, PI = quantiles empiriques des rendements historiques).

EXPLICATION. Comparer à RW évite de récompenser un modèle qui ne fait que suivre le prix. Skill positif = mieux que « ne rien faire ».

EXEMPLE. Les tests poolés portent sur des *différences de skill* (daily – weekly) par origine, pas sur les métriques brutes.

Coefficient de variation (CV) inter-graines

DÉFINITION. Écart-type divisé par la moyenne d'une métrique à *travers les graines* (`std / mean` sur les 5 graines). Mesure la reproductibilité d'un modèle : petit CV = résultats stables d'une graine à l'autre.

EXPLICATION. Sert à deux choses : (1) justifier la marge du TOST (l'ordre de grandeur de la variabilité inter-graines, 0,8–7,3 % sur le RMSE) ; (2) montrer que NsDiff est plus reproductible que TSDiff (~3,7× plus stable).

EXEMPLE. Un CV(Winkler) faible sur les 5 graines dit « ce modèle donnera à peu près la même calibration quelle que soit la graine tirée ».

4.5 Les tests statistiques

Bootstrap par blocs par percentile & la p-value « p »

DÉFINITION (EXACTE, `PAIRED_TEST.PY:23-76`). Aucune p-value n'est paramétrique. On rééchantillonne 10 000 fois la série chronologique des différences *par origine*, en **blocs de 3 origines consécutives** (`block_length=3` , `n_boot=10000`), et :

$p = 2 \times (\text{fraction des 10 000 moyennes rééchantillonnées tombant du côté de zéro OPPOSÉ à la moyenne observée, plafonnée à 1})$

EXPLICATION. p répond à : « si le vrai écart moyen était nul, quelle fraction des rééchantillonnages produirait un écart au moins aussi extrême ? ». **Bilatérale, sans hypothèse de normalité ni d'indépendance** — d'où le bootstrap *par blocs* : les cibles W+2/W+3 se chevauchent d'une origine à l'autre, donc les différences sont autocorrélées, pas i.i.d. ; les blocs de 3 capturent cette structure.

EXEMPLE / CORRECTION. « $p < 0.0001$ » est *imprécis*. Les valeurs brutes sont exactement `p = 0.0` . Le plus petit p non nul vaut $2/10\,000 = 2 \times 10^{-4}$. L'énoncé correct est **$p < 2 \times 10^{-4}$** , pas $p < 10^{-4}$: on ne descend pas plus fin sans augmenter `n_boot` .

p bilatérale vs unilatérale

DÉFINITION. *Bilatérale* : teste « différent de zéro » dans les deux sens (le facteur 2 dans la formule). *Unilatérale* : teste une seule direction (« A meilleur que B »), sans le facteur 2.

EXPLICATION. Le p du bootstrap est bilatéral et *plafonné à 1.0* ; ce plafonnement l'empêche d'être redivisé en un p unilatéral fiable. Le TOST, qui a besoin de tests unilatéraux, utilise donc son propre calcul (`return_boot_means=True` renvoie les moyennes brutes pour ça).

EXEMPLE. C'est la raison de la modification opt-in `paired_block_bootstrap_test(return_boot_means=False)` : exposer les moyennes bootstrap pour le TOST sans changer le comportement par défaut.

effective_n (puissance réelle)

DÉFINITION. Le nombre de blocs à peu près indépendants : `effective_n = n // block_length` (`paired_test.py:73`).

EXPLICATION. Comme les origines voisines se chevauchent, la puissance réelle est bien inférieure au n nominal. Reporter `effective_n` empêche de lire $n=30$ comme 30 points indépendants.

EXEMPLE. 30 origines, blocs de 3 → `effective_n = 10` ; ~15 sur le bloc de test conforme ; 13 en mensuel. Le docstring : « treat p-values as optimistic ; only read strong, robust effects as real ».

Seuil $\alpha=0,05$ & correction de Holm

DÉFINITION. Convention du dépôt : `significant_at_05` ($p < 0,05$). Holm-Bonferroni ajuste ce seuil quand on fait *plusieurs* tests, pour contrôler les faux positifs de famille (`holm_correction` , `pooled_analysis.py:178`).

EXPLICATION. $\alpha=0,05$ ne suffit pas ici. **(1) Tests multiples** : 30 cellules, 12 tests poolés sans correction → ~1,5 faux positif par famille de 30 ; Holm sur les 12 → seuil effectif $\approx 0,05/12 \approx 0,004$. **(2) Puissance faible** : un p juste au-dessus de 0,05 ne démontre rien.

EXEMPLE (GRILLE DE LECTURE). $p \approx 2 \times 10^{-4}$ → survit à toute correction = **résultat**. p entre 0,004 et 0,05 (ex. crypto/Winkler W+3, $p=0,0378$) → ne survit pas à Holm = **signal à confirmer**. $p > 0,05$ ne se lit *pas* comme absence d'effet — c'est ce que le TOST permet d'affirmer.

Test de Kupiec (POF, proportion of failures)

DÉFINITION. Test du rapport de vraisemblance sur la couverture : $H_0 =$ « la vraie couverture vaut p » (0,95). Vérifié `pooled_analysis.py:263` .

EXPLICATION. Il suppose des essais de Bernoulli **i.i.d.** — *optimiste* ici (origines corrélées). Rapporté *en complément*, à côté du verdict bootstrap par blocs (le chiffre fiable).

EXEMPLE. Kupiec compte, par actif, combien de graines rejettent à 5 % — utile pour raconter « combien de fois la couverture est significativement fausse », jamais comme verdict principal.

Test de Christoffersen (couverture conditionnelle, LR_ind)

DÉFINITION. Étend Kupiec en testant, en plus de la couverture *inconditionnelle*, l'**indépendance** des violations (une violation ne doit pas prédire la suivante) — la couverture conditionnelle. Sa composante d'indépendance est le `LR_ind` .

EXPLICATION. p -values *optimistes* (violations supposées i.i.d.), et `LR_ind` mécaniquement biaisé vers le rejet quand les cibles W+2/W+3 se chevauchent (dépendance mécanique). Rapporté en complément, jamais comme verdict.

EXEMPLE. C'est l'une des « briques neuves » (Kupiec + Christoffersen + TOST) du chantier de consolidation ; dans ce checkout, seul le prédécesseur Kupiec est présent (`pooled_analysis.py`).

TOST (Two One-Sided Tests) — équivalence, non-réfutation, « non conclusif »

DÉFINITION. Procédure qui teste l'**équivalence** plutôt que la différence : deux modèles sont déclarés équivalents si l'écart tient dans une marge fixée *a priori* ($\pm 5\%$ de RMSE relatif), via deux tests unilatéraux. Le TOST travaille sur le *ratio de RMSE*.

EXPLICATION. Trois issues à distinguer : *équivalent* (écart prouvé petit) ; *significativement différent* (écart prouvé) ; *non conclusif* (ni l'un ni l'autre — à `effective_n=30` il ne faut pas le lire « interchangeable »). Un $p > 0,05$ ordinaire n'est qu'une *non-réfutation* (« pas prouvé différent »), pas une équivalence ; seul le TOST conclut positivement.

EXEMPLE. Le TOST transforme « indistinguishable » en « équivalent » sur la précision daily de 4 actifs/5 à W+1 et W+2 — équivalence *établie*. Réserve : il porte sur la performance *attendue*, pas sur la garantie d'un run individuel.

MCS & SPA (au-delà du duel, face à GARCH)

DÉFINITION. MCS (Model Confidence Set, Hansen-Lunde-Nason 2011) : le sous-ensemble de modèles statistiquement indissociables du meilleur, à un niveau α . SPA (Superior Predictive Ability, Hansen 2005) : teste H_0 « aucun modèle ne bat le benchmark » (GARCH(1,1)). Code : `mcs.py`.

EXPLICATION. Le duel n'est qu'un test par paires ; MCS/SPA répondent « qui est dans le peloton de tête » et « quelqu'un bat-il vraiment GARCH ». Même bootstrap par blocs, mêmes indices de temps appliqués à tous les modèles (préserve la corrélation inter-modèles).

EXEMPLE. Rejet SPA vs GARCH(1,1) toujours à 0/75 : aucun run `NsDiff` ne dépasse significativement GARCH — le mur tient.

4.6 Conventions de lecture

Les trois niveaux : descriptif / significatif single-seed / significatif poolé multi-graines

DÉFINITION. **Descriptif** = un chiffre, jamais testé. **Significatif à graine fixe** = test sur une seule graine (ce que fait le dashboard). **Significatif poolé multi-graines** = métrique moyennée sur les 5 graines origine par origine, puis testée.

EXPLICATION. Le pooling multi-graines réduit le bruit Monte-Carlo sans créer de points de donnée : l'unité d'inférence reste l'origine ($n=90$, `effective_n=30`). Ce qui est testé ainsi est la performance *attendue d'un run à graine tirée au hasard* — pas celle d'un ensemble des 5 graines (ça, c'est la tâche 6, §4.7).

EXEMPLE. « RMSE 274,0 » seul = descriptif ; le même avec un `p bootstrap` et un `effective_n` = testé. Ne jamais confondre les deux.

Poolé par origine (`run_pooled_test`)

DÉFINITION. On regroupe par origine (`cutoff_date`) : moyenne cross-sectionnelle des différences de skill de tous les (modèle, actif) contribuant à cette origine, puis bootstrap par blocs sur cette série. Vérifié `dashboard_d7_w1.py:701`.

EXPLICATION. « Blocs = origines consécutives » : chaque élément du vecteur testé est une origine, jamais un mélange.

EXEMPLE. Le label « `weekly_native_significantly_better` » est générique de cette fonction partagée ; réutilisé en mensuel, il faut le traduire « régime C significativement meilleur » (pas un bug).

Pooling par classe & dédoublement « taux » (ZN=F + TLT)

DÉFINITION. Avant le pooling, les deux obligations corrélées (ZN=F et TLT) sont fusionnées en **une** contribution « taux » par (modèle, origine) — la moyenne des deux, pas deux voix indépendantes (`build_pooled_series`).

EXPLICATION. Compter ZN=F et TLT comme deux points indépendants gonflerait artificiellement l'échantillon obligataire (ils bougent ensemble). Le dédoublement évite ce double comptage.

EXEMPLE. Quand TLT est absent (dérive yfinance), la classe « taux » ne repose plus que sur ZN=F : à lire avec réserve, ne pas conclure sur les obligations à partir d'un seul actif.

Single-seed vs ensemble multi-graines

DÉFINITION. *Single-seed* : un seul tirage de graine (le dashboard `oos` = seed 42, déterministe, comparable aux autres modèles). *Multi-graines* : 5 graines (42–46), pour tester la robustesse ou pour *ensembler* (tâche 6).

EXPLICATION. Le protocole multi-graines n'est **jamais écrit dans `tracking.db`** — artefacts JSON isolés. C'est le cœur du chantier « dashboard » : passer de single-seed/50 à multi-graines/200 suppose de régénérer *tous* les modèles au même budget, sinon la comparabilité mutuelle casse.

EXEMPLE. `nsdiff_daily_weekly_multiseed.py` relance NsDiff sur 42–46, mêmes origines, mêmes budgets, et produit une table de CV — sans toucher la DB.

Robustesse / stabilité multi-graines (verdict stable)

DÉFINITION. Un verdict est *stable* s'il ne change pas d'une graine à l'autre (le test par cellule rend le même résultat sur les 5 graines). Non-négociable du corpus : aucun verdict n'est présenté sans sa stabilité multi-graines.

EXPLICATION. Un verdict qui bascule de graine en graine (proche de 0) n'est pas fiable ; un verdict 5/5 (ou 25/25 sur graines×actifs) l'est. C'est distinct du p : le p mesure la significativité sur une graine, la stabilité mesure la reproductibilité du verdict.

EXEMPLE. Mensuel-natif : NsDiff bat TSDiff 25/25 (5 actifs × 5 graines) — le résultat le plus robuste du corpus. Le sous-régime B daily, lui, est bruité graine à graine.

4.7 Recalibration & sélection

Recalibration conforme (split conforme, par niveau)

DÉFINITION. Méthode qui corrige la couverture d'un modèle *sans ré-entraîner* : découpe chronologique calibration/test, mesure d'un *score de non-conformité* par point, puis élargissement (ou resserrement) des bandes d'un facteur k calibré. Code : `tsdiff_recalibrate.py`.

EXPLICATION. Le score est *side-aware* (asymétrique) : $s = (y - \text{médiane}) / (\text{haut} - \text{médiane})$ si $y \geq \text{médiane}$, sinon $(\text{médiane} - y) / (\text{médiane} - \text{bas})$ — sans hypothèse gaussienne. Le facteur k = quantile empirique des scores de calibration, corrigé fini-échantillon ($\lceil (n+1) \cdot L \rceil / n$), et **par (actif, niveau), jamais global**.

EXEMPLE. À $n_{\text{calib}} \approx 45$, le quantile retenu est la $\sim 44^{\text{e}}$ statistique d'ordre sur 45 (97,8^e percentile) : *franchement conservateur*, il sur-corrige (couverture 0,99–1,00, largeur de SPY qui double). La sous-couverture du daily est corrigable, mais pas par ce levier.

Score de non-conformité & facteur k (warping du nuage)

DÉFINITION. Le *score de non-conformité* mesure à quel point une réalisation est « surprenante » pour la distribution prédite (distance à la médiane, normalisée par la demi-largeur du bon côté). Le *facteur k* déforme (*warp*) le nuage pour que ses quantiles atteignent la couverture visée.

EXPLICATION. $k > 1$ élargit, $k < 1$ resserre. Le warping applique un $k(L)$ interpolé à travers les niveaux pour que le nuage recalibré reproduise exactement les quantiles corrigés — et que le CRPS reste calculable dessus.

EXEMPLE. Un k médian de 1,5 sur ETH-USD signifie « les bandes de TSDiff devaient être élargies de 50 % pour couvrir correctement sur le bloc de calibration ».

Scénario A vs B (contrôle d'équité)

DÉFINITION. Deux façons de tester la recalibration. *Scénario A* : on recalibre le daily **seul** et on le compare au weekly non recalibré. *Scénario B* : on recalibre **les deux** (contrôle d'équité).

EXPLICATION. Le scénario A est trompeur lu seul : le weekly « gagne » non parce qu'il est meilleur, mais parce qu'on a infligé au daily une sur-correction en largeur qu'on n'a pas infligée au weekly — et le Winkler facture la largeur. Le scénario B, symétrique, ramène à « indistinguable ».

EXEMPLE. Scénario A : « weekly significativement meilleur, $p=0$ » ; scénario B : « indistinguable, $p=0,114$ ». La différence est l'artefact d'équité.

Sweep d'époques (argmin CRPS de validation)

DÉFINITION. Sélection du nombre d'époques par validation : pour chaque (actif, modèle), on retient le budget qui minimise le CRPS *sur le bloc de validation* (`select_epochs` , `epoch_sweep.py:274`).

EXPLICATION. Garde-fou explicite : seul `crps_val` alimente ce choix — le bloc de test n'est jamais touché. C'est ce qui a corrigé le bras daily de TSDiff (§3.1) : candidats 10/20/40, retenu 20 dans 19 cellules/25.

EXEMPLE. C'est aussi ce garde-fou qui a été *violé* côté weekly (§3.2) : la validation d'époques de TSDiff-W chevauchait la grille de test — fuite déclarée, non corrigée, mais favorable à TSDiff donc conservatrice.

Ensemble multi-graines (tâche 6) — le meilleur rapport qualité/prix

DÉFINITION. On *concatène* les 5 nuages de 200 tirages en un nuage de 1 000 (mélange des 5 prédictives), puis on lit point et bandes dessus avec la même formule qu'un run simple.

EXPLICATION. Concaténer élargit le nuage quand les graines sont en désaccord (effet recherché) ; moyenner les *bornes* l'aurait au contraire lissé. Coût : **aucun refit** (les 5 fits existent déjà). C'est la config de production visée.

EXEMPLE. Sur le daily, l'ensemble améliore le Winkler sur 7 cellules/15 (contre 2 pour le weekly) et monte la couverture des deux côtés — « gratuitement », là où le conformal sur-corrige.

4.8 Les concepts du mécanisme

Rapport paramètres / données (paramètres par fenêtre)

DÉFINITION. Nombre de paramètres du modèle divisé par le nombre de fenêtres d'entraînement disponibles. Plus il est élevé, plus le modèle risque de mémoriser au lieu de généraliser.

EXPLICATION. La variable qui explique TOUT le motif du duel. « Changer de régime » ne change pas que l'horizon : sur SPY, ça change l'échantillon d'entraînement (478 → 2 420 fenêtres), donc le ratio de TSDiff (525 → 104 paramètres/fenêtre).

EXEMPLE. TSDiff weekly 525/fenêtre = sur-paramétrisation → dominé par NsDiff ; TSDiff daily 104/fenêtre → égalité sur le point.

Sur-paramétrisation & sur-entraînement

DÉFINITION. *Sur-paramétrisation* : trop de paramètres pour la quantité de données. *Sur-entraînement* : trop d'époques, le modèle finit par mémoriser le train.

EXPLICATION. Les deux se conjuguent chez TSDiff : sur-paramétré en weekly/mensuel, et sensible au sur-entraînement dès qu'on lui donne beaucoup de pas de gradient (daily à budget weekly, §3.1).

EXEMPLE. Au-delà de ~20 époques, la variance résiduelle de TSDiff-D tend vers zéro — largeur du PI 12,4 % → 0,12 % entre 10 et 80 époques.

Collapse de diffusion (échantillonneur effondré)

DÉFINITION. État où le modèle génératif cesse de produire une vraie loi : tous les tirages se rapprochent, la dispersion tend vers zéro, l'intervalle devient minuscule et la couverture s'effondre.

EXPLICATION. Mode d'échec caractéristique de TSDiff au sur-apprentissage : rien ne paramètre sa dispersion, donc c'est la première chose qui casse. Signature : intervalles ~40× trop étroits, Cov95 0,00–0,10, étiquette code `structurally under-calibrated / coverage_95: 0.0`.

EXEMPLE. Le sous-régime B mensuel de TSDiff montre exactement ce symptôme (Cov95 7,5–30 %) : ses cellules ne doivent pas être lues comme une comparaison de point propre (§1.3).

4.9 Protocole, code & reproductibilité

`tracking.db` & `source='oos'` (live / backtest_rolling / oos)

DÉFINITION. La base SQLite qui stocke les prédictions de tous les modèles. La colonne `source` distingue les pistes : `'oos'` (out-of-sample, la piste officielle du dashboard), `'live'`, `'backtest_rolling_*'` (pistes isolées d'autres chantiers).

EXPLICATION. Le dashboard ne lit que `source='oos'`. Écrire un modèle sous un `source_tag` isolé ne le ferait pas apparaître. Non-négociable : `tracking.db` n'est *jamais écrit* par les runs multi-graines (artefacts JSON isolés) — pour garder la piste `oos` propre et comparable.

EXEMPLE. NsDiff est devenu visible sur le dashboard en insérant ses lignes daily+weekly en `source='oos'` sur les origines exactes des 6 autres modèles (`insert_oos_predictions` , `upsert idempotent`).

Le dashboard D7/W1

DÉFINITION. La page de synthèse (`dashboard_d7_w1.py` → HTML autonome) qui compare, par modèle, le régime daily (D7) et weekly (W1) à W+1, en single-seed 42, `n_samples=50`.

EXPLICATION. C'est la vitrine « production » — mais en config non représentative (50 tirages, une graine). Le migrer vers multi-graines/200 est précisément l'étude en cours (priorité 2).

EXEMPLE. Il affiche « cellules » (par modèle × actif, testées) et un « agrégat poolé » (par classe d'actifs).

Prix gelés (frozen prices, `.parquet`) & dérive yfinance

DÉFINITION. Les séries de prix sont téléchargées une fois, *gelées* dans des fichiers `.parquet` partagés, et lues identiquement par les deux modèles. La *dérive yfinance* : yfinance re-sert des cotes légèrement différentes d'un appel à l'autre (retraitement rétroactif des dividendes, $\sim 2 \times 10^{-7}$ de bruit relatif).

EXPLICATION. Geler les prix est un non-négociable : sinon les deux modèles ne verraient pas exactement la même donnée, et un fit de 40–80 époques amplifie $\sim 6 \times 10^{-6}$ de tels écarts. C'est aussi ce qui fait échouer TLT (le retraitement casse la vérification de dernier close).

EXEMPLE. TLT : `last_close mismatch` 86,59 vs 86,94 ($\sim 0,4$ % relatif) → bascule sur le cache offline (`offline_prices.py` , `DONNEE~1.XLS`), mais le cache s'arrête au dernier cutoff → TLT skip proprement.

Vérification bit-à-bit & déterminisme

DÉFINITION. *Bit-à-bit* : deux exécutions produisent des sorties **numériquement identiques** (0 écart). *Déterministe* : à graine fixée, le résultat est reproductible exactement.

EXPLICATION. C'est la preuve qu'un refactor ou un ajout opt-in n'a rien changé au comportement d'origine. Le duel s'y appuie partout : la machinerie de test partagée (`diffusion_headtohead.py`) est vérifiée à « 0 écart numérique », et le bras weekly se reproduit bit-à-bit après la correction du bras daily.

EXEMPLE. Correctif §3.1 vérifié sur seed42/SPY : largeur weekly 5,930 % → 5,930 %, échantillons identiques — le daily change, le weekly non.

Modifications opt-in / additives / rétrocompatibles

DÉFINITION. Des changements de code qui n'activent un nouveau comportement que si on le demande explicitement (un kwarg avec un défaut préservant l'ancien comportement). *Additif* = on ajoute, on ne casse rien.

EXPLICATION. Trois fichiers partagés ont été touchés ainsi, comportement par défaut strictement inchangé : `generate_nsdiff_asset(collect_samples=False)` , `build_enriched_pairs(horizon_unit="W+1")` , `paired_block_bootstrap_test(return_boot_means=False)` . C'est ce qui garantit que les résultats des autres chantiers restent valides.

EXEMPLE. `collect_samples=True` (opt-in) fait conserver le nuage complet pour le CRPS ; par défaut (False), rien ne change pour les appels existants.

Briques réutilisées vs réimplémentées · tests unitaires (invariants, entrées dégénérées)

DÉFINITION. *Réutiliser une brique* = importer et appeler une fonction existante déjà testée, plutôt que d'en réécrire une variante. *Test unitaire* = un test automatique d'une fonction ; un *invariant* = une propriété qui doit toujours tenir ; une *entrée dégénérée* = un cas limite (ensemble à 1 point, différences toutes nulles...).

EXPLICATION. Réutiliser évite d'introduire des divergences silencieuses : le bootstrap, le Winkler, le pooling, la recalibration sont tous importés, pas recodés. Seuls Kupiec/Christoffersen/TOST sont du code neuf — couverts par 19 tests unitaires (cas calculés à la main, entrées dégénérées, invariants).

EXEMPLE. `pytest` : 351 passed, 1 skipped (le skip = `properscoring not installed`, pré-existant et sans rapport). L'état vert avant ET après est un non-négociable.

5. Limites déclarées & l'étape manquante

Puissance. `effective_n=30` (90 origines, blocs de 3) pour le duel ; ≈ 15 sur le bloc de test conforme ; ≈ 13 par cellule en mensuel. Aucun « indistinguable » ne doit être lu comme une absence d'effet démontrée — sauf là où le TOST conclut explicitement à l'équivalence.

Tests multiples non corrigés. Beaucoup de cellules (5 actifs $\times$ 3 horizons $\times$ 2 régimes) ; les cases isolées à $p \approx 0,04$ (crypto/Winkler W+3) ne survivent pas à Holm et se lisent comme « signal à confirmer ».

Kupiec / Christoffersen optimistes. p-values χ^2 supposant des violations i.i.d. ; rapportées en complément, jamais comme verdict principal — le test de référence reste le bootstrap par blocs.

Asymétrie de protocole vs GARCH. GARCH est refit à chaque origine (déterministe, sans graine) ; NsDiff est train-once-forward et dépend d'une graine. La question posée est « un run NsDiff à graine tirée au hasard bat-il GARCH ? », pas « le meilleur NsDiff bat-il GARCH ? ».

CRPS/PIT absents en mensuel et sur le dashboard. Ces métriques exigent le nuage complet, non stocké dans ces pipelines ; la comparaison y porte sur RMSE (testé), Cov95, Winkler, direction.

L'étape manquante — et la plus importante pour trancher. Tout ce document mesure la *qualité prédictive et la calibration*. Il ne mesure pas la **valeur économique**. Un modèle parfaitement calibré peut n'avoir aucun edge : des intervalles larges et justes ne rapportent rien. Le seul test qui tranche « utilisable en trading » est un **backtest économique** — intervalles $\rightarrow$ décisions (sizing, VaR, signaux), coûts de transaction (0 € pour l'instant) et slippage, mesure de PnL / Sharpe / drawdown, NsDiff contre GARCH. Les résultats de calibration ci-dessus sont une condition *nécessaire* (un modèle mal calibré donnerait un sizing faux), pas *suffisante*. Le duel prédiction/calibration débroussaille le terrain ; il ne conclut pas la question du edge.

Document généré pour DeepEdgeBenchmark — synthèse détaillée et étendue du duel NsDiff vs TSDiff, ancrée sur le code du dépôt. Les valeurs numériques proviennent des notes `NOTE_duel_nsdiff_vs_tsdiff_budget_egal.md`, `NOTE_nsdiff_consolidation_daily_vs_weekly.md` et `NOTE_compare_tsdiff_nsdiff_monthly.md` ; les définitions de calcul sont vérifiées dans `paired_test.py`, `dashboard_d7_w1.py`, `crps_metrics.py`, `pooled_analysis.py`, `tsdiff_recalibrate.py`, `epoch_sweep.py` et `mcs.py`.